Software Architecture of SIGA

1 Software Architecture of SIGA

SIGA is designed as a modular GPU-accelerated isogeometric analysis framework for nonlinear and higher-gradient computational mechanics. The architecture separates the program into five major layers: geometry and spline-basis construction, degree-of-freedom and boundary-condition management, GPU-based residual and tangent assembly, sparse linear/nonlinear solution, and post-processing. This separation allows the same geometric and discretization infrastructure to be reused by several physical models, including nonlinear elasticity, strain-gradient elasticity, electroelasticity, and flexoelectricity.

Figure 1 shows the main data flow in SIGA. The user first defines knot vectors, control points, material parameters, boundary conditions, and solver options. These inputs are converted into patch-level and multipatch data structures. The discretization layer constructs the spline basis, performs degree elevation or refinement, and builds the global degree-of-freedom mapping. The assembly layer then transfers the required geometry, basis, quadrature, and sparse-system information to the GPU. At each Newton iteration, SIGA evaluates the residual vector and tangent matrix on the GPU, stores the resulting sparse system in compressed sparse row format, and calls a linear solver backend. After convergence, the solution and derived quantities are reconstructed as spline fields and written to visualization files.

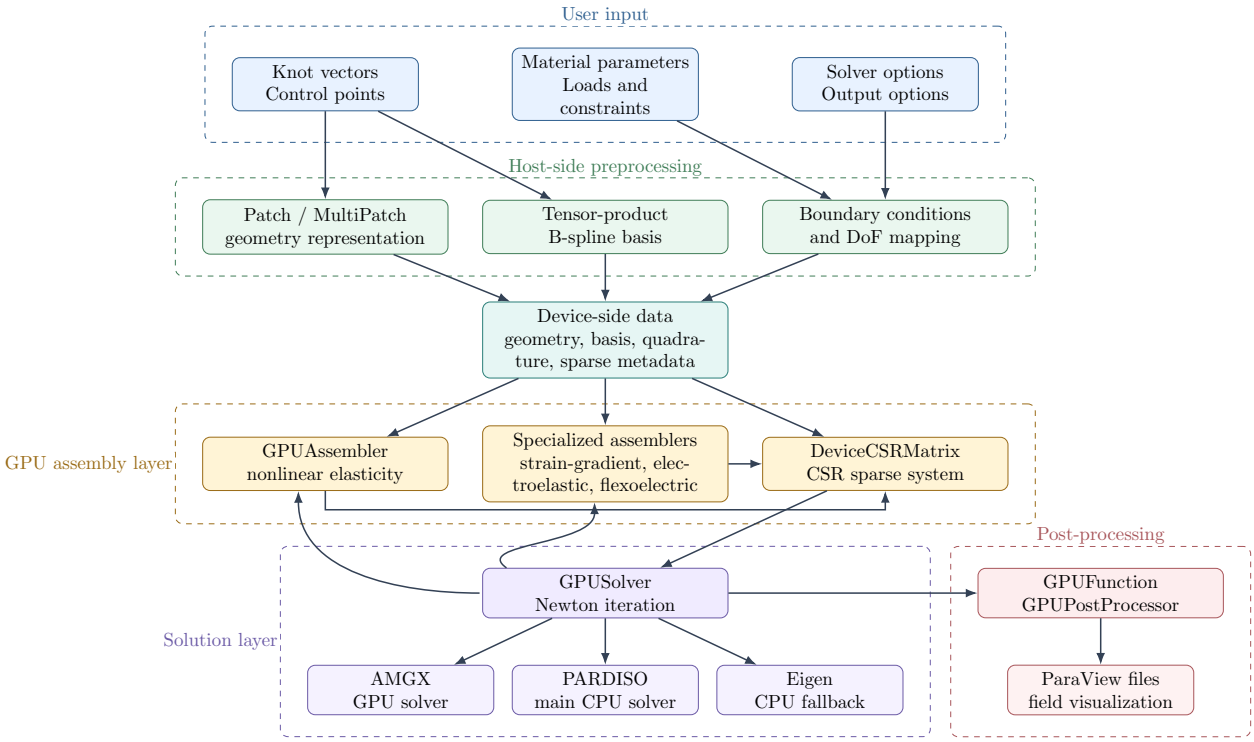

Figure 1: Overall software architecture and data flow of SIGA. Geometry, basis, and boundary-condition information are prepared on the host and transferred to device-side data structures. Residuals and tangent matrices are assembled on the GPU, stored in CSR format, and solved inside a Newton iteration. Converged solutions and derived fields are exported for visualization.

1.1 Geometry and Basis Layer

The geometry layer of SIGA is built around patch-based spline representations. A single patch stores a tensor-product spline basis together with its control points, while a multipatch object stores multiple patches and their topological relationships. This structure allows SIGA to represent both single-patch and multipatch domains in two and three dimensions. During preprocessing, the multipatch geometry is analyzed to identify interfaces and boundaries. These topological relationships are then used to construct conforming degree-of-freedom mappings and to apply boundary conditions.

The discretization layer is based on tensor-product B-spline basis functions. Each basis object stores the knot vectors, polynomial orders, element structure, active basis functions, and quadrature information associated with a patch. Uniform refinement, knot insertion, and degree elevation are handled at the basis level. The same basis infrastructure is used for geometry representation, displacement approximation, electric-potential approximation, and post-processing fields. This design avoids duplicating discretization logic across different physics modules.

1.2 Degree-of-Freedom and Boundary-Condition Management

SIGA separates the geometric description from the algebraic unknowns. After the patch and basis objects are constructed, a degree-of-freedom mapper assigns global indices to free unknowns and boundary indices to constrained unknowns. This mapping is used to impose Dirichlet conditions and to construct sparse row and column layouts. For coupled-field problems, separate field blocks

can be created for displacement components and scalar electric potential. The resulting block structure is used by the sparse system and by solver strategies such as coupled or Schur-type solution procedures.

Boundary conditions are represented independently from the assembly kernels. This makes it possible to reuse the same geometry and basis data for different loading cases, such as displacement control, Neumann loading, follower moments, and coupled electro-mechanical constraints. In nonlinear simulations, prescribed values can be refreshed between load steps without reconstructing the entire geometry or basis.

1.3 GPU Assembly Layer

The central computational component of SIGA is the GPU assembly layer. The base assembly class stores device-side geometry, basis, quadrature, sparse-system, and boundary-condition data. During each Newton iteration, the assembler evaluates basis functions and derivatives at quadrature points, computes deformation measures and constitutive responses, and assembles the residual vector and tangent matrix. The base assembler is used for finite-strain nonlinear elasticity, while specialized assemblers extend the same interface to higher-gradient and coupled-field problems.

The derived assembly classes implement additional physics-specific quantities. The strain-gradient assembler requests second derivatives of the displacement basis and introduces a material length-scale parameter. This is particularly well suited to isogeometric discretizations because spline bases provide higher inter-element continuity than standard low-order finite elements. The electroelastic and flexoelectric assemblers introduce an additional scalar electric-potential field and assemble coupled mechanical-electrical tangent blocks. Figure 2 summarizes this organization.

1.4 Sparse Matrix Representation

The sparse algebraic system is stored in compressed sparse row format on the device. During initialization, the sparsity pattern is constructed from the degree-of-freedom mapping and active basis connectivity. During nonlinear iterations, only the matrix values and residual entries need to be updated. This distinction between sparsity-pattern construction and value assembly is important for nonlinear problems, because the same pattern is reused over multiple Newton iterations and load steps.

SIGA uses a device CSR matrix abstraction to store row pointers, column indices, and matrix values. This representation is compatible with GPU-based solvers and can also be transferred to host-side sparse matrices when CPU solver backends are used. In the current implementation, PARDISO is the primary CPU sparse solver, while Eigen is kept as the fallback path when PARDISO support is not available. The sparse matrix layer therefore acts as an interface between the GPU assembly kernels and the available linear solver backends.

1.5 Nonlinear Solver Layer

The nonlinear solution procedure is implemented using a Newton iteration. At each iteration, the current solution is passed to the assembler, which computes the residual vector and tangent matrix. The linearized correction is then obtained using one of the available solver backends. When AMGX is enabled, SIGA can solve the sparse system using a GPU-based iterative solver. When AMGX is disabled or unsuitable for a particular problem, the system can be transferred to the host and solved with PARDISO as the main CPU direct solver. If PARDISO is not available in the build, SIGA falls back to Eigen-based sparse solvers. This design provides flexibility between fully GPU-oriented workflows and robust CPU-based fallback paths.

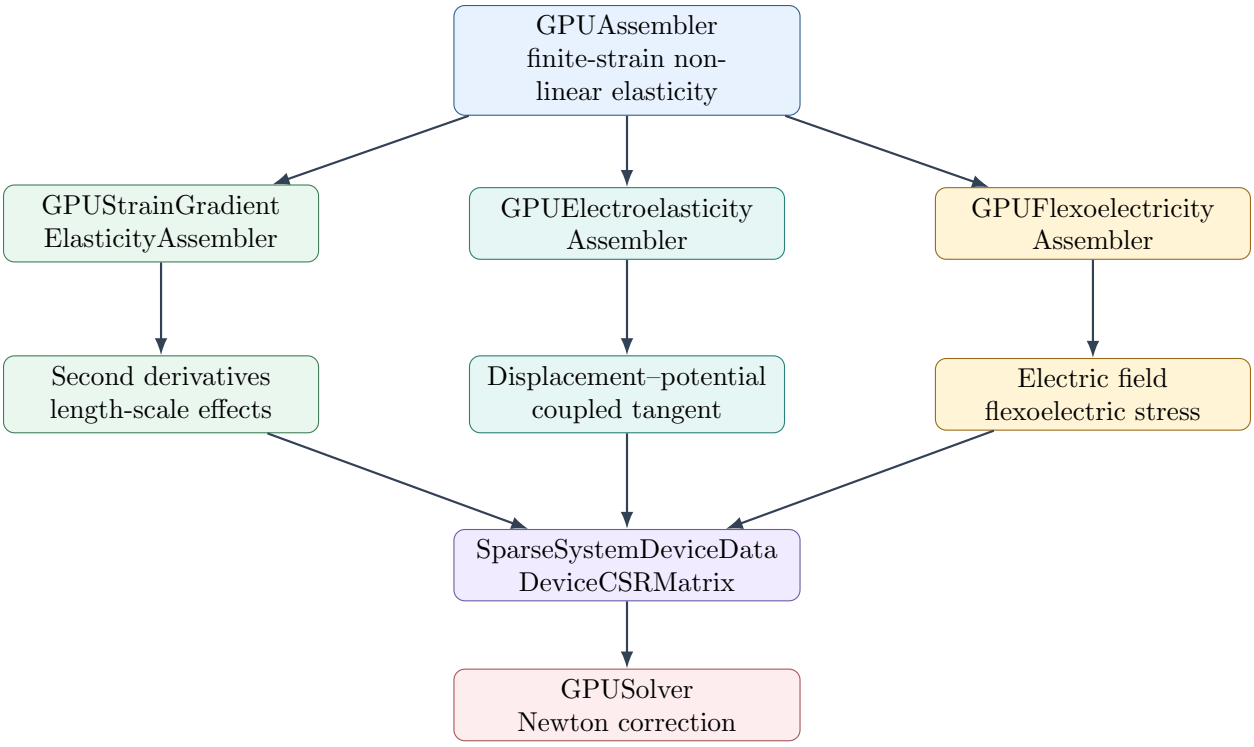

Figure 2: Assembler hierarchy in SIGA. The base GPU assembler provides the common nonlinear mechanics assembly infrastructure, while specialized assemblers extend the formulation to strain-gradient elasticity, electroelasticity, and flexoelectricity. All assemblers generate residual and tangent contributions in a common sparse-system format used by the Newton solver.

The solver layer also stores convergence information, including residual norms, update norms, iteration counts, and timing data. These quantities are useful both for controlling nonlinear simulations and for reporting performance results. In addition to standard Newton solution, the solver provides eigenvalue and stability-related functionality, which is used in buckling and instability examples.

1.6 Post-Processing Layer

After convergence, SIGA reconstructs physical fields from the solution vector. Displacement, stress, deformation gradient, strain-gradient quantities, electric potential, and electric field can be represented as spline-based functions. These fields are evaluated at visualization points and exported through the post-processing module. The output is written in ParaView-compatible formats, allowing the user to inspect undeformed and deformed geometries, stress distributions, electric fields, and higher-gradient quantities.

The post-processing layer is deliberately separated from the solver and assembler. This makes it possible to use the same computational core for verification studies, performance benchmarks, and application simulations while changing only the output fields required by each example.

1.7 Design Advantages

The main advantage of the SIGA architecture is that geometry, basis construction, physical assembly, sparse algebra, nonlinear solution, and visualization are separated into reusable modules.

This design has several practical benefits. First, new physical models can reuse the same patch, basis, boundary-condition, and sparse-system infrastructure. Second, GPU kernels can be specialized for expensive model-specific operations while preserving a common solver interface. Third, the same nonlinear solver can be used across classical elasticity, strain-gradient elasticity, and coupled electro-mechanical formulations. Finally, the common post-processing interface allows different simulations to export comparable field quantities for visualization and analysis.

Overall, the architecture is intended to support extensibility and performance simultaneously. The host side handles symbolic and topological operations such as geometry construction, refinement, and degree-of-freedom mapping, while the GPU side handles the computationally intensive quadrature, residual assembly, tangent assembly, sparse matrix updates, and field recovery operations.

2 Parallel Assembly Strategy, Multi-GPU Support, and Memory Management

The dominant computational cost in SIGA comes from repeated residual and tangent-matrix assembly during Newton iterations. This cost is especially significant for high-order isogeometric discretizations because each element contains multiple quadrature points and each quadrature point couples all active spline basis functions. SIGA therefore performs the most expensive quadrature, constitutive evaluation, and sparse assembly operations on the GPU. The assembly strategy is organized around three design principles: element-level parallelism, reuse of precomputed basis and quadrature data, and memory-aware streaming over groups of elements.

2.1 Parallel Assembly Decomposition

At each Newton iteration, SIGA first reconstructs the current displacement field from the solution vector and prescribed degrees of freedom. The geometry, basis values, basis derivatives, quadrature points, sparse-system metadata, and material parameters are already stored in device-side data structures. The assembly procedure then proceeds in three main phases:

1. Gauss-point precomputation, where deformation measures, geometry Jacobians, integration weights, stresses, and material tangent quantities are evaluated.
2. Tangent-matrix assembly, where element-level contributions are accumulated into the global sparse matrix.
3. Residual-vector assembly, where internal-force and external-force contributions are accumulated into the global right-hand side.

For a displacement discretization with N_e elements, N_D active basis functions per element, and spatial dimension d, the tangent assembly can be viewed as a parallel loop over element-basis pairs

$$(e, i, j), \qquad e = 1, \ldots, N_e, \quad i, j = 1, \ldots, N_D.$$

Each such task computes a local block

$$\mathbf{K}_{ij}^{e} \in \mathbb{R}^{d \times d},$$

which contains all component couplings between basis functions N_i and N_j. The residual assembly is similarly decomposed over element-basis pairs (e, i), producing local vectors

$$\mathbf{R}_{i}^{e} \in \mathbb{R}^{d}.$$

Within each CUDA block, threads iterate over quadrature points and accumulate the corresponding local matrix or residual contribution. After all quadrature contributions are accumulated, the local block is inserted into the global sparse system.

Figure 3 illustrates this decomposition. The important point is that SIGA does not assign an entire element matrix to a single serial thread. Instead, the element stiffness computation is broken into many block-level tasks indexed by active basis-function pairs. This strategy exposes a large amount of parallelism for high-order spline elements.

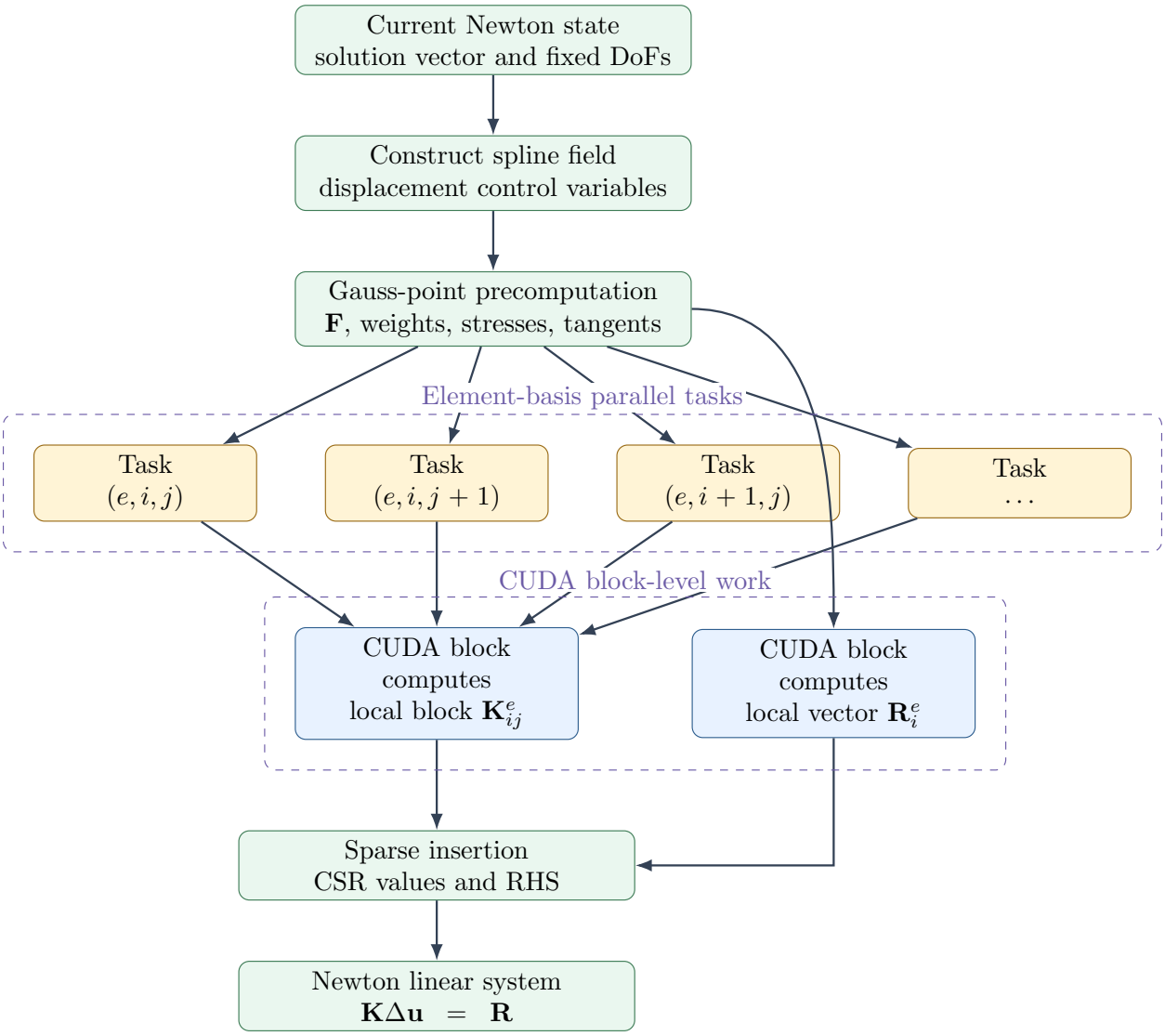

Figure 3: Parallel decomposition of SIGA's GPU assembly. The tangent matrix is decomposed into element-basis pair tasks (e, i, j), while the residual vector is decomposed into element-basis tasks (e, i). Each CUDA block accumulates quadrature-point contributions for a small local block before inserting the result into the global sparse system.

The same decomposition is used for higher-gradient formulations, but with additional Gauss-point data. In strain-gradient elasticity, for example, the local tangent block contains contributions from both first-gradient and second-gradient terms. The block-level computation uses precomputed

geometry inverses, weights, geometry Hessians, stress-like quantities, and material tangent blocks. The local tangent contribution includes contractions of the form

$$\mathbf{B}_i^T \frac{\partial \mathbf{P}_1}{\partial \mathbf{F}} \mathbf{B}_j, \qquad \mathbf{B}_i^T \frac{\partial \mathbf{P}_1}{\partial \nabla \mathbf{F}} \mathbf{D}_j, \qquad \mathbf{D}_i^T \frac{\partial \mathbf{P}_2}{\partial \nabla \mathbf{F}} \mathbf{D}_j,$$

where $\mathbf{B}$ denotes the first-gradient operator and $\mathbf{D}$ denotes the second-gradient operator. This structure is well suited to IGA because the higher continuity of spline basis functions allows second derivatives to be evaluated inside patches.

2.2 Sparse Assembly and Fixed-Degree-of-Freedom Treatment

SIGA stores the global tangent matrix in compressed sparse row format. The sparsity pattern is built from the active basis connectivity and degree-of-freedom mapping, while the matrix values are updated during each Newton iteration. During insertion, each local contribution is mapped from patch-local active basis indices to global row and column indices. Free degrees of freedom contribute directly to the CSR matrix. Contributions associated with eliminated fixed degrees of freedom are redirected to the right-hand side, which allows Dirichlet constraints to be handled consistently inside the same assembly path.

This design avoids forming dense element matrices on the host. Instead, element-level contributions are accumulated directly into device-side sparse data structures. The sparse assembly layer therefore acts as the interface between local quadrature computations and the global Newton system.

2.3 Streaming Assembly

For very large problems, storing all temporary Gauss-point data for the entire domain can exceed available GPU memory. SIGA addresses this issue through streaming assembly. Instead of assembling the whole domain in one pass, the element range is divided into chunks. Each chunk contains a contiguous set of elements, the associated Gauss points, and the corresponding local sparse row window. The assembly kernels are then applied chunk by chunk.

For each chunk, SIGA performs the following operations:

1. determine the element range and Gauss-point range;
2. determine the affected sparse row range;
3. allocate or reuse temporary Gauss-point buffers;
4. evaluate Gauss-point quantities for the chunk;
5. assemble the chunk-local tangent and residual;
6. accumulate the chunk contribution into the global CSR matrix and RHS vector.

Figure 4 shows one streaming step. The full element set is kept as a host-side chunk schedule, but the GPU temporary-memory window contains only the active chunk, its Gauss-point data, and the affected sparse row window. After that chunk is accumulated into the global CSR matrix and RHS vector, the temporary buffers are released or reused for the next chunk. The same code path can run on a single GPU or across multiple GPUs. When only one GPU is used, streaming mainly reduces peak memory usage. When multiple GPUs are used, the chunks are distributed across devices.

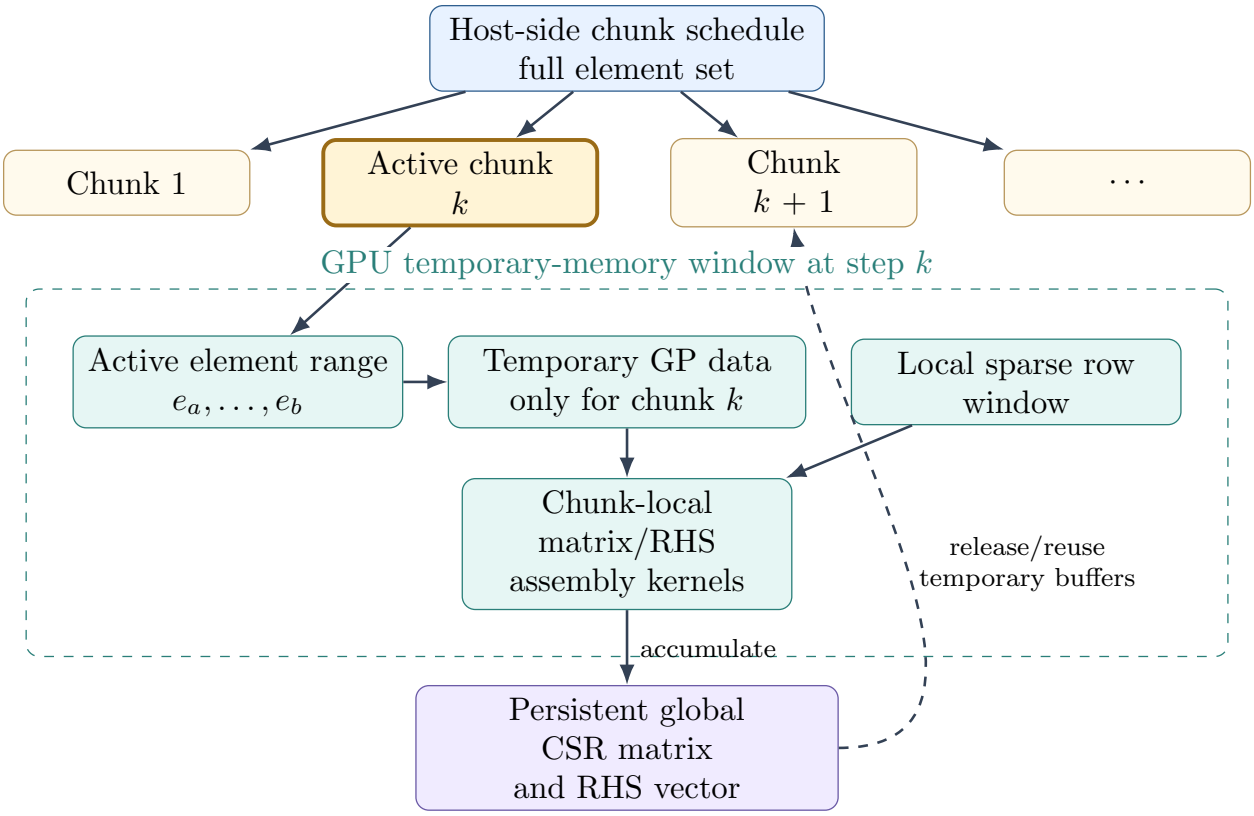

Figure 4: Streaming assembly strategy. The full element set is divided into a host-side chunk schedule, but only the active chunk k, its Gauss-point data, and its sparse row window occupy temporary GPU buffers at a given streaming step. After the chunk-local matrix and residual are accumulated into the persistent global CSR matrix and RHS vector, the temporary buffers are released or reused for the next chunk.

2.4 Multi-GPU Assembly

Multi-GPU assembly in SIGA extends the streaming strategy by distributing chunks over all usable CUDA devices. SIGA first identifies the primary CUDA device and then scans the visible devices. A secondary device is used only if peer access is available in both directions between the primary device and the secondary device. Peer access is enabled before the assembly begins. This check avoids assigning work to GPUs that cannot exchange data efficiently with the primary device.

The primary GPU stores the global CSR matrix and RHS vector. Secondary GPUs receive replicated static input data, such as geometry, basis, Gauss-point tables, material data, and sparse metadata. During each Newton iteration, dynamic data such as displacement control points and fixed degrees of freedom are refreshed. Each secondary GPU assembles into a local sparse output buffer, which contains only the matrix-value and RHS entries associated with its assigned sparse row window. After each round of chunk assembly, the secondary output buffers are reduced into the global CSR matrix and RHS on the primary GPU.

Figure 5 summarizes the multi-GPU data flow.

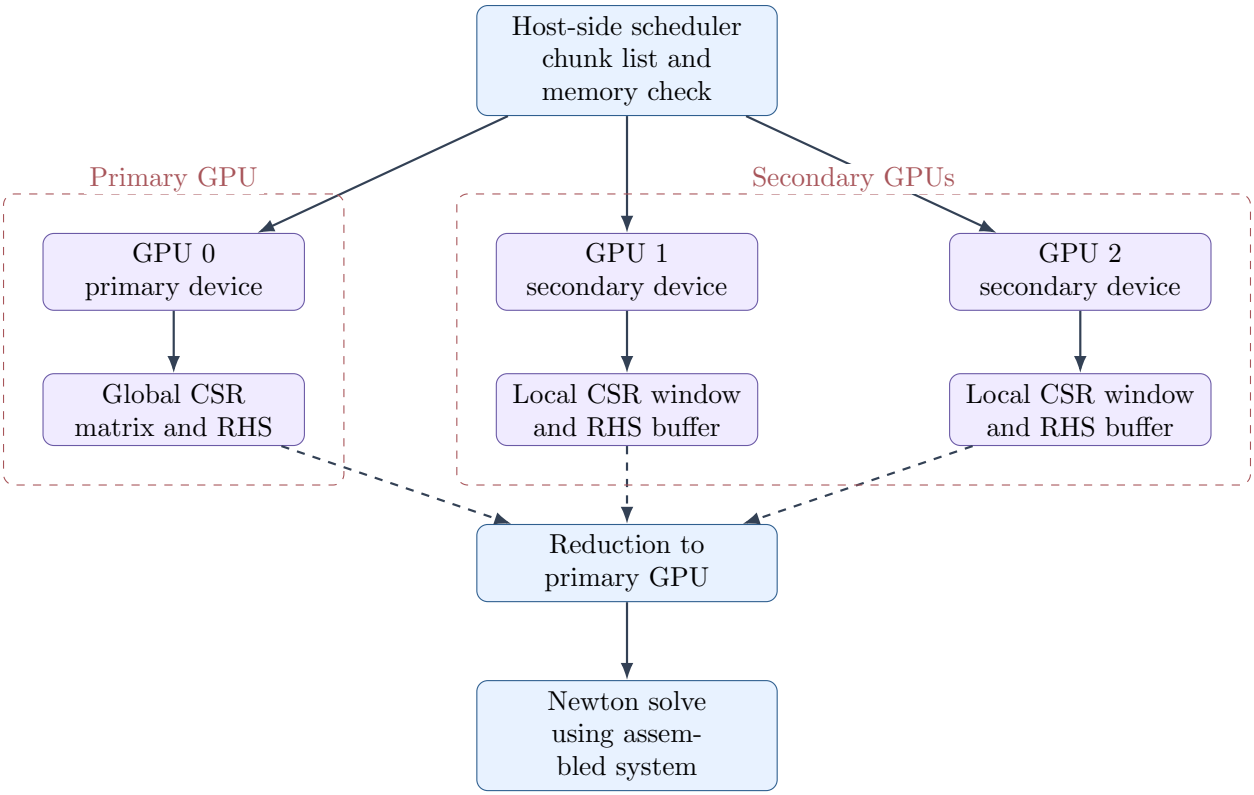

Figure 5: Multi-GPU assembly in SIGA. The primary GPU owns the global sparse system. Secondary GPUs assemble assigned chunks into local sparse output buffers. After each assembly round, local matrix values and residual entries are reduced into the primary CSR matrix and RHS vector.

The chunk distribution is round-robin. If n_g usable GPUs are available, chunk k is assigned to device

$$g(k) = k \bmod n_g.$$

This simple policy keeps the implementation robust and avoids complicated load-balancing overhead. Since the chunks are generated with a bounded maximum number of elements, the amount of work per chunk is approximately controlled by the user-specified or automatically reduced chunk size.

Within each round, SIGA launches chunk work on all devices that have an available chunk. The implementation uses one host thread per active device in a round. Each host thread sets its CUDA device, launches the Gauss-point precomputation, matrix assembly, and RHS assembly kernels for the assigned chunk, and synchronizes the device. After all device threads complete, the primary device performs the reduction of secondary sparse output buffers.

2.5 Memory-Aware Chunk Sizing

The streaming and multi-GPU assembly path is memory-aware. SIGA estimates the amount of temporary memory required on each device before launching the assembly. The estimate includes:

- local matrix-value output buffer;
- local RHS output buffer;

- material-parameter array;
- Gauss-point temporary data;
- sparse metadata and local CSR row window;
- replicated static input data;
- replicated fixed-degree-of-freedom data.

Let M_g denote the currently free memory on GPU g, and let $\eta \in (0, 1]$ denote the memory safety fraction. SIGA requires

$$M_g^{\mathrm{req}} \leq \eta M_g.$$

If the requested chunk size violates this inequality, SIGA automatically reduces the maximum number of elements per chunk and rebuilds the schedule. This procedure is repeated until the schedule fits within the memory target or until even a one-element chunk cannot fit. The safety fraction is controlled by the environment variable

`SIGA_GPU_MEMORY_FRACTION`.

The initial chunk size can be specified using

`SIGA_GPU_CHUNK_ELEMENTS`.

If no chunk size is specified, SIGA starts from a one-shot partition in which the elements are divided approximately evenly among the usable assembly devices.

Figure 6 illustrates the memory-management loop.

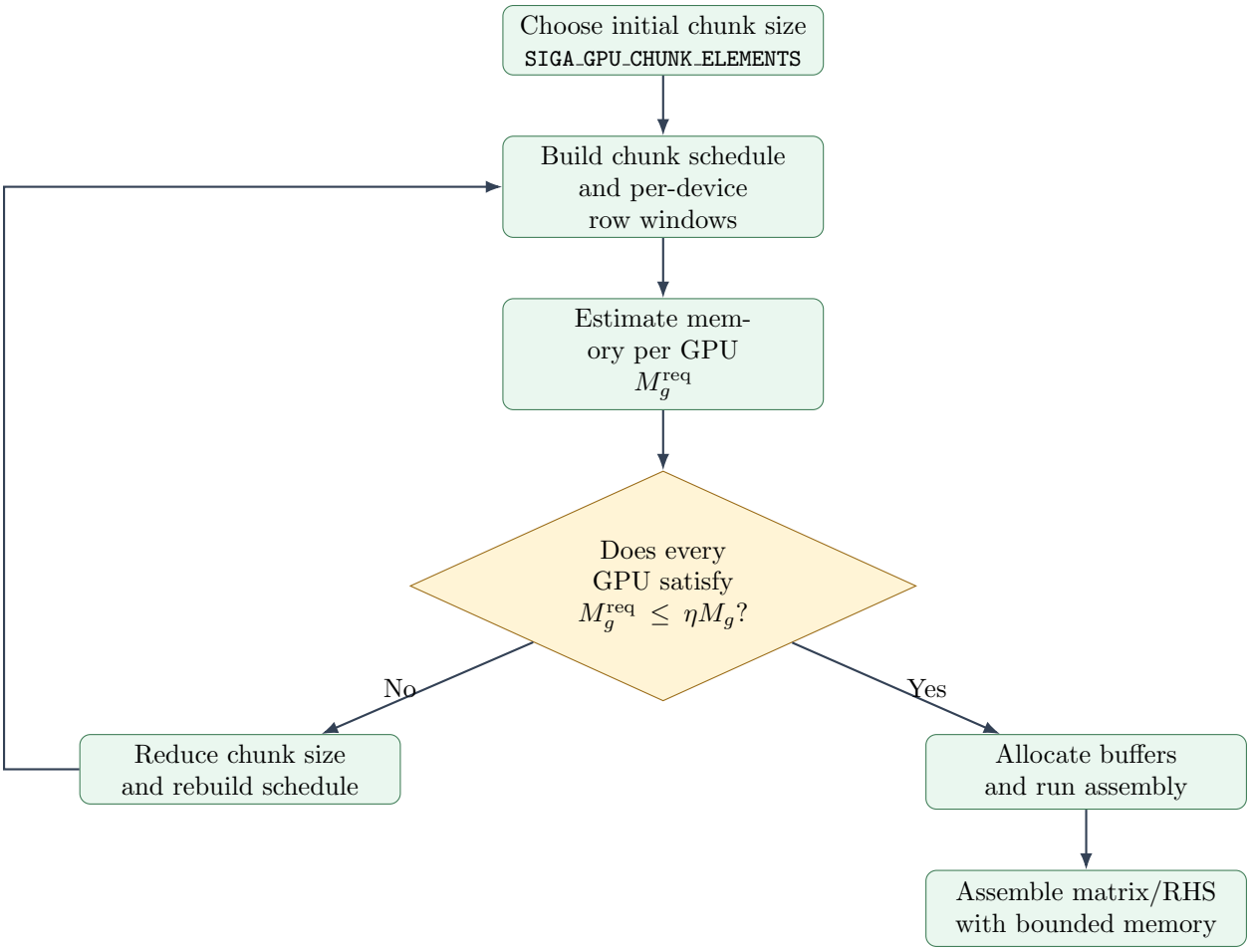

Figure 6: Memory-aware chunk-size selection. SIGA estimates the required temporary memory on each GPU and compares it with a safety fraction of the currently available memory. If the schedule does not fit, the element chunk size is reduced and the schedule is rebuilt.

The memory-management strategy is particularly important for higher-order IGA. For a degree-p tensor-product basis in d dimensions, the number of active basis functions per element scales approximately as

$$N_D \sim (p+1)^d.$$

The local tangent work therefore scales with N_D^2, and the temporary quadrature data also increase with the number of Gauss points and derivative quantities stored per point. In higher-gradient problems, the memory footprint is even larger because the formulation requires second derivatives, geometry Hessians, and additional constitutive tangent blocks. Streaming assembly allows SIGA to handle these cases without requiring all temporary element and Gauss-point data to be resident simultaneously.

2.6 Configurable Assembly Modes

The assembly behavior can be controlled at runtime without recompilation. The most relevant environment variables are:

- `SIGA_MULTIGPU_ASSEMBLY`: enables multi-GPU assembly.

- `SIGA_GPU_STREAMING_ASSEMBLY`: enables streaming assembly. By default, streaming is enabled when multi-GPU assembly is enabled.
- `SIGA_GPU_CHUNK_ELEMENTS`: sets the requested maximum number of elements per chunk.
- `SIGA_GPU_MEMORY_FRACTION`: sets the safety fraction used in memory fitting.
- `SIGA_GPU_REPLICATE_INPUTS`: controls whether secondary GPUs receive replicated geometry, basis, and quadrature input data.
- `SIGA_GPU_LOCAL_CSR_ASSEMBLY`: controls whether secondary GPUs assemble into local CSR row windows.
- `SIGA_GPU_MATRIX_TIMING`: enables detailed timing output for assembly phases.
- `SIGA_GPU_MEMORY_REPORT`: enables memory diagnostics for assembly buffers.

These options make it possible to use the same executable in several modes. For small problems, a one-shot single-GPU assembly may be sufficient. For large problems that exceed the memory available for temporary Gauss-point data, streaming can be enabled on one GPU. For still larger problems or for machines with multiple peer-accessible GPUs, multi-GPU streaming distributes chunks across devices and reduces their local sparse outputs to the primary device.

2.7 Discussion

The proposed parallel assembly strategy reflects the structure of isogeometric discretizations. High-order spline elements produce expensive element-level operations because many active basis functions interact over each element. SIGA exposes this work by parallelizing over elements, active basis pairs, quadrature points, and field components. Temporary Gauss-point quantities are reused between matrix and residual assembly, while the global sparse system remains in CSR format throughout the Newton iteration.

The multi-GPU implementation follows a primary-secondary model. The primary GPU owns the global sparse matrix and residual vector, while secondary GPUs assemble assigned chunks into local output buffers. This design avoids concurrent writes from multiple devices into the same global CSR array and gives a clear reduction point after each chunk round. The memory-aware chunking mechanism further ensures that the assembly can be adapted to different GPU memory capacities by reducing the number of elements processed in each chunk.

Overall, SIGA's assembly strategy is designed to balance three competing requirements: high parallel throughput, compatibility with sparse Newton solvers, and robustness for large high-order simulations whose temporary memory requirements may exceed the capacity of a single GPU.

3 Smallest Eigenpair Computation for Stability Analysis

In nonlinear mechanics simulations, loss of stability is commonly associated with the loss of positive definiteness or near-singularity of the tangent stiffness matrix. SIGA therefore provides routines for computing the critical eigenpair of the assembled tangent operator. The implementation supports two cases: a single-field displacement formulation and a coupled displacement–electric-potential formulation. In both cases, the objective is not to solve a dynamic modal problem with a mass matrix, but rather to identify the critical static tangent eigenmode associated with material or structural instability.

3.1 Single Displacement Field Formulation

For the pure mechanical formulation, the unknown vector contains only displacement degrees of freedom,

$$\mathbf{q} = \mathbf{u}.$$

After the current Newton state has been reconstructed, SIGA assembles the tangent matrix

$$\mathbf{K}_{uu} = \frac{\partial \mathbf{R}_u}{\partial \mathbf{u}},$$

where $\mathbf{R}_u$ denotes the mechanical residual. The critical eigenpair is obtained from

$$\mathbf{K}_{uu}\boldsymbol{\psi} = \lambda \boldsymbol{\psi}.$$

Here, λ is the tangent eigenvalue and $\boldsymbol{\psi}$ is the corresponding displacement eigenmode. A vanishing or negative value of λ indicates loss of local stability of the current equilibrium path.

SIGA computes this eigenpair using an inverse iteration. Starting from a normalized vector $\boldsymbol{x}_0$, the algorithm repeatedly solves

$$\mathbf{K}_{uu}\boldsymbol{y}_k = \boldsymbol{x}_k,$$

then normalizes

$$\boldsymbol{x}_{k+1} = \frac{\boldsymbol{y}_k}{\|\boldsymbol{y}_k\|}.$$

The inverse eigenvalue is estimated from

$$\mu_k = \boldsymbol{x}_k^T \boldsymbol{y}_k,$$

and the tangent eigenvalue is approximated by

$$\lambda_k = \frac{1}{\mu_k}.$$

The iteration stops when the change in λ_k is sufficiently small. After convergence, SIGA recomputes the eigenvalue using the Rayleigh quotient

$$\lambda = \boldsymbol{x}^T \mathbf{K}_{uu} \boldsymbol{x},$$

and returns $\boldsymbol{x}$ as the displacement eigenvector.

Figure 7 summarizes the single-field computation.

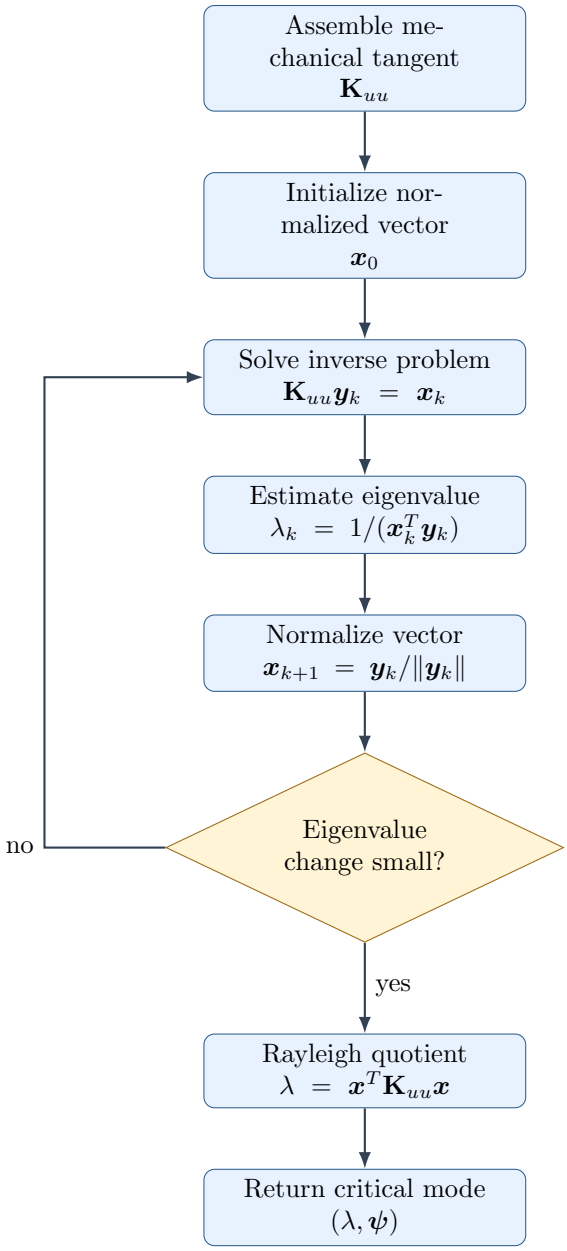

Figure 7: Smallest critical eigenpair computation for the single displacement-field formulation. SIGA uses inverse iteration on the assembled mechanical tangent matrix and evaluates the final eigenvalue by a Rayleigh quotient.

The linear solve inside the inverse iteration can use different backends. When AMGX support is enabled, the tangent matrix is uploaded in CSR format and the inverse-iteration solve is performed using the AMGX solver. If AMGX fails to converge, SIGA falls back to a CPU sparse direct solve. When AMGX is not enabled, the matrix is converted to a host-side sparse matrix and solved using either PARDISO, when available, or Eigen's sparse solver backend. In incremental simulations, the previously computed eigenvector can be reused as the initial vector, which reduces the number of inverse-iteration steps needed after small load increments.

3.2 Coupled Displacement–Electric-Potential Formulation

For electroelastic and flexoelectric problems, the unknown vector contains both displacement and electric-potential degrees of freedom,

$$\mathbf{q} = \begin{bmatrix} \mathbf{u} \\ \boldsymbol{\phi} \end{bmatrix}.$$

The linearized coupled tangent system has the block form

$$\mathbf{H} = \begin{bmatrix} \mathbf{H}_{uu} & \mathbf{H}_{u\phi} \\ \mathbf{H}_{\phi u} & \mathbf{H}_{\phi\phi} \end{bmatrix},$$

where $\mathbf{H}_{uu}$ is the mechanical tangent block, $\mathbf{H}_{\phi\phi}$ is the electric-potential block, and the off-diagonal blocks describe electro-mechanical coupling.

For stability analysis, SIGA computes a condensed mechanical eigenpair. The electric-potential increment is eliminated from the coupled system by assuming that the electric field remains equilibrated with respect to a given mechanical perturbation. From the second block row,

$$\mathbf{H}_{\phi u}\Delta\mathbf{u} + \mathbf{H}_{\phi\phi}\Delta\boldsymbol{\phi} = \mathbf{0},$$

we obtain

$$\Delta\boldsymbol{\phi} = -\mathbf{H}_{\phi\phi}^{-1}\mathbf{H}_{\phi u}\Delta\mathbf{u}.$$

Substituting this expression into the first block row gives the condensed mechanical tangent

$$\widehat{\mathbf{H}}_{uu} = \mathbf{H}_{uu} - \mathbf{H}_{u\phi}\mathbf{H}_{\phi\phi}^{-1}\mathbf{H}_{\phi u}.$$

The critical coupled stability problem is then written as

$$\widehat{\mathbf{H}}_{uu}\boldsymbol{\psi}_u = \lambda\boldsymbol{\psi}_u.$$

After the mechanical eigenvector $\boldsymbol{\psi}_u$ is found, the associated electric-potential eigenvector is recovered by

$$\boldsymbol{\psi}_\phi = -\mathbf{H}_{\phi\phi}^{-1}\mathbf{H}_{\phi u}\boldsymbol{\psi}_u.$$

SIGA does not need to explicitly form $\widehat{\mathbf{H}}_{uu}^{-1}$ for the inverse iteration. Instead, at each inverse-iteration step, it solves the full coupled system

$$\begin{bmatrix} \mathbf{H}_{uu} & \mathbf{H}_{u\phi} \\ \mathbf{H}_{\phi u} & \mathbf{H}_{\phi\phi} \end{bmatrix} \begin{bmatrix} \boldsymbol{y}_{u,k} \\ \boldsymbol{y}_{\phi,k} \end{bmatrix} = \begin{bmatrix} \boldsymbol{x}_{u,k} \\ \mathbf{0} \end{bmatrix}.$$

The displacement part $\boldsymbol{y}_{u,k}$ is equivalent to applying the inverse of the condensed operator,

$$\boldsymbol{y}_{u,k} = \widehat{\mathbf{H}}_{uu}^{-1}\boldsymbol{x}_{u,k}.$$

The mechanical vector is then normalized,

$$\boldsymbol{x}_{u,k+1} = \frac{\boldsymbol{y}_{u,k}}{\|\boldsymbol{y}_{u,k}\|},$$

and the eigenvalue is estimated as

$$\lambda_k = \frac{1}{\boldsymbol{x}_{u,k}^T\boldsymbol{y}_{u,k}}.$$

After convergence, SIGA evaluates the final condensed Rayleigh quotient,

$$\lambda = \boldsymbol{x}_u^T \widehat{\mathbf{H}}_{uu} \boldsymbol{x}_u,$$

and recovers the electric-potential eigenvector by back-substitution.

Figure 8 illustrates the coupled-field procedure.

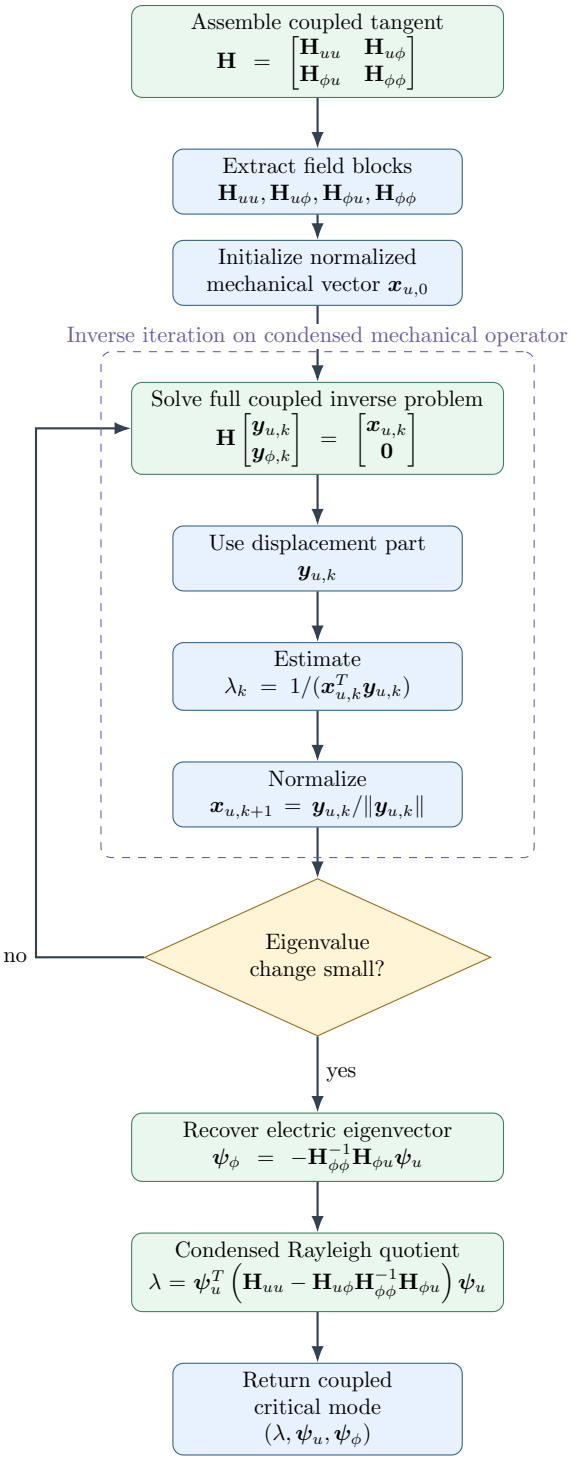

Figure 8: Smallest condensed mechanical eigenpair computation for the coupled displacement–electric-potential formulation. The inverse iteration is applied to the Schur-complement mechanical operator by solving the full coupled system with a mechanical right-hand side. The electric-potential eigenvector is recovered by back-substitution after convergence.

This condensed formulation has two advantages. First, the returned eigenvalue corresponds directly to the mechanical stability of the electro-mechanically equilibrated system. Second, the

method avoids explicitly inverting the electric-potential block. The inverse action of $\mathbf{H}_{\phi\phi}$ is applied only through sparse linear solves. In the current implementation, the coupled eigenpair routine works on the host-side sparse matrix and uses PARDISO when enabled; otherwise, it uses Eigen's sparse solver backend.

3.3 Interpretation of the Computed Eigenpair

For both formulations, the computed eigenpair should be interpreted as a tangent-stability indicator. When the smallest critical eigenvalue approaches zero, the current equilibrium configuration is close to a bifurcation, buckling point, or other instability. The associated eigenvector gives the perturbation direction. In the single-field case, this perturbation consists only of displacement components. In the coupled-field case, the perturbation consists of both a mechanical part and a compatible electric-potential part,

$$\boldsymbol{\psi} = \begin{bmatrix} \boldsymbol{\psi}_u \\ \boldsymbol{\psi}_\phi \end{bmatrix}.$$

This coupled mode can be used to perturb the converged solution and continue the simulation along a post-critical branch.

The method described here is different from a vibration or modal-frequency analysis. No mass matrix is included, and the eigenproblem is not a generalized eigenproblem. The computed value is an eigenvalue of the current tangent operator, or of the condensed tangent operator in the coupled case. It is therefore most appropriate for quasi-static stability, buckling, and bifurcation studies.